

The report is delivered — committed to branch `discovery/security-report-20260723T184156` at `docs/discovery/06-security.md` and opened as [PR #5](#). Below is the required chat output, verbatim from the saved report.

OVERALL CODEBASE RATING — SECURITY

High Risk

Driven by a critical, systemic lack of authentication/authorization on every API route, amplified by wildcard CORS, an `extract()` variable-injection sink, and committed default credentials.

EXECUTIVE SUMMARY

The Klearcom platform is a full-stack monolith and this review covered all three layers — the Laravel PHP backend, the React/TypeScript frontend, and the Node/Express `dev-api`. The dominant finding is systemic: **there is no authentication or authorization anywhere** — every REST route (discovery, connect, dashboard, MongoDB transcripts/diagnostics, and SSE streams) is fully public, and object IDs are trusted straight from the client with no ownership check, so any anonymous caller can read, create, and mutate all tenant data. This is compounded by a **wildcard CORS policy** (`allowed_origins => ['*']` in Laravel and a bare `app.use(cors())` in Express) that lets any website read those responses, a **PHP `extract()` on raw request input** in the legacy reporting controller (variable injection), debug mode forced on, and weak database credentials committed to `.env.example`, `docker-compose.yml`, and the CI workflow. The frontend is comparatively healthy — React's JSX auto-escaping means no XSS sinks, no client-side secrets, and no JWT-in-`localStorage` were observed — but it ships with no CSP or security headers and talks to the API over plain HTTP. Dependency hygiene is good on paper (React 19, Vite 6, Express 4.21, Laravel 12, MongoDB driver 6/2 are all current), but no `npm audit`/`composer audit` or SAST step exists in CI to catch future CVEs. Overall posture is **High Risk**, driven by the total absence of access control.

6.1 Security Benchmark Ratings

#	Security KPI	Target	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Critical Vulnerabilities	0	0	1	>1	1 (no access control)	MODERATE
H2	High Vulnerabilities	0	<5	5–10	>10	2 (CORS <code>*</code> , <code>extract()</code>)	GOOD
H3	Medium Vulnerabilities	low	<20	20–50	>50	5	GOOD
H4	Vulnerability Density	<0.5/KLOC	<0.5	0.5–1.0	>1.0	~2.4/KLOC (8 findings / ~3.4 KLOC)	HIGH RISK
H5	OWASP Top 10 Compliance	>95%	>95%	80–95%	<80%	20% clean (2/10)	HIGH RISK

H6	Critical/High Vulnerable Deps	0	0	1	>1	0	GOOD
H7	Outdated Dependencies	<10%	<10%	10–25%	>25%	~0% (all majors current)	GOOD
H8	End-of-Life Dependencies	0	0	1–5	>5	0	GOOD

6.3 OWASP Top 10 (2021) Coverage

#	Category	Verdict	Evidence / Note
6.1	Broken Access Control	CRITICAL	No auth on any route; IDOR via <code>findOrFail(\$id)</code> — <code>routes/api.php</code> , all controllers, <code>dev-api/src/server.js</code>
6.2	Cryptographic Failures	MEDIUM	Committed <code>secret</code> / <code>root</code> DB creds; API served over cleartext <code>http://</code> ; no TLS/HSTS enforcement
6.3	Injection	HIGH	<code>extract()</code> on <code>\$request->all()</code> (variable injection) — <code>LegacyReportController.php</code> , <code>LegacyDataMapper.php</code> . SQLi clean (Eloquent)
6.4	Insecure Design	MEDIUM	No rate limiting; unbounded unvalidated <code>bulk-import</code> — <code>dev-api/src/server.js</code>
6.5	Security Misconfiguration	HIGH	Wildcard CORS <code>*</code> ; <code>APP_DEBUG=true</code> ; no CSP/security headers — <code>config/cors.php</code> , <code>docker-compose.yml</code> , <code>index.html</code> , nginx conf
6.6	Vulnerable & Outdated Components	CLEAN	All npm/composer majors current; 0 EOL. Gap is no automated scan (see 6.12), not versions
6.7	Identification & Authentication Failures	HIGH	No authentication mechanism, no MFA, no lockout — no <code>users</code> password/auth flow implemented anywhere
6.8	Software & Data Integrity Failures	CLEAN	Deps use <code>^</code> ranges but lockfiles present and installed in CI; no unsigned auto-update/deserialization path
6.9	Security Logging & Monitoring Failures	MEDIUM	No audit log for access events; empty exception handler; <code>console.error</code> only; no alerting
6.10	Server-Side Request Forgery (SSRF)	CLEAN	No server-side HTTP request built from user-supplied URL/host
6.11	Other Security Reviews	CLEAN	No file upload, path traversal, SSTI, or mass-assignment beyond validated <code>create()</code> observed
6.12	DevSecOps Security Assessment	MEDIUM	CI (<code>ci.yml</code>) has no SAST/ <code>npm audit</code> / <code>composer audit</code> ; secrets committed; runtime <code>composer install</code> in Docker

6.5 Actions Required

Finding	Action	Rating	Priority
---------	--------	--------	----------

Broken Access Control / no authentication (6.1, 6.7)	Add auth middleware (Sanctum/JWT) + ownership policies to every route group; remove IDOR via <code>findOrFail</code>	HIGH RISK	CRITICAL
Wildcard CORS (6.5)	Replace <code>'*'</code> with an explicit origin allow-list in <code>config/cors.php</code> and <code>cors()</code> options	HIGH RISK	HIGH
<code>extract()</code> variable injection (6.3)	Remove <code>extract()</code> ; read named params explicitly and validate	MODERATE	HIGH
Debug mode + committed secrets (6.2, 6.5)	Disable debug outside local; move DB creds to secrets manager and rotate	MODERATE	MEDIUM
No rate limiting / unbounded bulk-import (6.4)	Add <code>throttle</code> / <code>express-rate-limit</code> and validate + size-cap payloads	MODERATE	MEDIUM
Missing CSP/security headers + cleartext HTTP (6.2, 6.5 / FS5)	Add CSP + security headers (nginx/meta) and enforce HTTPS/HSTS	MODERATE	MEDIUM
No security logging/monitoring (6.9)	Add audit logging for access/auth events + alerting on repeated failures	MODERATE	MEDIUM
No SAST/dependency scan in CI (6.12)	Wire <code>npm audit</code> / <code>composer audit</code> + a SAST step into <code>ci.yml</code> ; add branch protection	MODERATE	MEDIUM